Asistente turístico inteligente de Tenerife

1. Introducción y objetivos

El objetivo de este proyecto es desarrollar un asistente turístico conversacional especializado en Tenerife, capaz de responder preguntas sobre lugares de interés, actividades y recomendaciones turísticas, así como consultar previsiones meteorológicas.

Para ello, se han integrado tres funcionalidades principales: un sistema RAG (*Retrieval-Augmented Generation*) que utiliza una guía turística como fuente documental mediante una herramienta, un sistema de memoria para mantener conversaciones multiturno y otra herramientas externas para conocer la climatología de un día concreto.

La solución se ha desarrollado íntegramente en Python, dentro de un Jupyter Notebook, buscando un equilibrio entre sencillez, funcionalidad y facilidad de reproducción.

2. Diseño y decisiones técnicas

2.1. Elección del modelo de lenguaje: Google Gemini

Se ha elegido **Google Gemini** principalmente por dos motivos: la disponibilidad de acceso gratuito a su API, sujeto a los límites del servicio, y la familiaridad adquirida durante la práctica de la asignatura.

Además, el SDK ***google-genai*** permite implementar generación de texto, embeddings, streaming y llamadas a herramientas dentro de un mismo ecosistema, reduciendo la complejidad del proyecto.

El modelo generativo está configurado por defecto como ***gemini-3.5-flash-lite***, mientras que para la generación de embeddings se utiliza ***gemini-embedding-001***.

Los parámetros de generación elegidos son:

- **Temperatura = 0.3:** valor relativamente bajo para favorecer respuestas consistentes y reducir la variabilidad. Al tratarse de un asistente turístico basado en información documental, se prioriza la precisión frente a la creatividad. Este parámetro no elimina completamente el riesgo de respuestas incorrectas.
- **Máximo de tokens de salida = 1000:** limita la extensión de las respuestas y evitar un consumo innecesario de recursos. Se considera suficiente para responder a las consultas turísticas habituales.

2.2. Implementación del sistema RAG

Se ha utilizado un sistema RAG para que el asistente pueda fundamentar sus respuestas en la guía turística proporcionada, en lugar de depender exclusivamente del conocimiento del modelo.

Extracción y fragmentación del documento

El PDF se procesa mediante ***PyMuPDF***, conservando la página de procedencia de cada fragmento.

Para dividir el texto se utiliza ***RecursiveCharacterTextSplitter***, con los siguientes parámetros:

- **chunk_size = 700:** proporciona suficiente contexto en cada fragmento y evitar recuperar bloques demasiado extensos
- **chunk_overlap = 100:** permite conservar parte del contexto cuando una explicación queda dividida entre dos fragmentos

Se ha optado por una fragmentación recursiva que intenta separar el texto por doble salto de línea, salto de línea simple, punto, coma, etc. en ese orden. Esta se ha considerado la mejor opción ya que permite que es los *chunks* contengan frases o párrafos enteros, es la forma más natural de dividir un texto.

Embeddings y almacenamiento vectorial

Los fragmentos se convierten en vectores mediante el modelo ***gemini-embedding-001***. Se distingue entre ***RETRIEVAL_DOCUMENT*** para los documentos y ***RETRIEVAL_QUERY*** para las consultas.

Se ha elegido **FAISS** como motor de búsqueda vectorial por su sencillez de integración con Python y porque permite búsquedas eficientes sin necesitar un servidor de base de datos independiente.

Concretamente, se utiliza *IndexFlatIP* junto con la normalización L2 de los vectores. Esta combinación permite utilizar el producto escalar como medida de similitud coseno.
Se recuperan por defecto los tres fragmentos más similares a cada pregunta. Este número busca aportar contexto suficiente sin introducir demasiada información potencialmente irrelevante.

Por último, se conserva la página y el identificador de cada fragmento para que el modelo pueda citar las fuentes utilizadas.

2.3. Implementación de herramientas mediante function calling

Se han desarrollado dos herramientas:

- **search_internal_docs_tool:** recupera información relevante de la guía turística mediante el sistema RAG.
- **get_weather_tool:** consulta la API Open-Meteo para obtener temperaturas máximas y mínimas, precipitaciones y condiciones meteorológicas de una fecha determinada.

Se ha elegido la API de *Open-Meteo* porque ofrece acceso sin necesidad de una clave API para el uso no comercial. Además, proporciona los datos estructurados en JSON, facilitando su procesamiento, y de manera gratuita.
La consulta meteorológica utiliza como referencia las coordenadas de Santa Cruz de Tenerife y la zona horaria de Canarias. Por tanto, los resultados no representan necesariamente las condiciones de toda la isla.

Aunque Gemini permite la ejecución automática de funciones Python, se ha optado por declarar las herramientas mediante JSON Schema y gestionar manualmente su ejecución, proporcionando un mayor control sobre las llamadas, los argumentos, los resultados y los errores.
Se utiliza un registro de funciones ***registry*** que relaciona el nombre solicitado por el modelo con su implementación Python.
También se registran los intentos de ejecución en ***tool_logs***, permitiendo posteriormente comprobar qué herramientas se han utilizado y si se han producido errores.

2.4. Memoria conversacional y control de tokens

Para mantener conversaciones multiturno se almacena el historial mediante una lista de objetos, conservando los mensajes del usuario, las respuestas del modelo y las llamadas a herramientas.

Se ha establecido un límite de **5000 tokens** para controlar el tamaño del historial elegido como medida preventiva para limitar el crecimiento del contexto y el consumo de tokens. No representa el límite máximo de contexto admitido por Gemini.
Cuando se supera el umbral, el sistema solicita al modelo un resumen de la conversación anterior, conservando información relevante como preferencias, fechas, lugares y recomendaciones.

Su principal inconveniente es que el resumen puede omitir información relevante, ya que cuando se supera el limite la conversación se somete a un modelo que lo resume y puede omitir contexto relevante.

2.5. Streaming de respuestas

Se ha incorporado la generación de respuestas mediante ***generate_content_stream***.

Esta decisión busca mejorar la experiencia del usuario, mostrando el contenido progresivamente sin tener que esperar a que el modelo termine de generar toda la respuesta.
El streaming se ha integrado con el ***function calling manual,*** permitiendo identificar las solicitudes de herramientas, ejecutarlas y continuar generando la respuesta con la información obtenida.

3. Evaluación del sistema

Para evaluar el funcionamiento del asistente se han diseñado pruebas reproducibles sobre los componentes principales.

3.1. Evaluación del RAG

Se ha preparado un conjunto de preguntas relacionadas con diferentes aspectos turísticos de Tenerife, como miradores, gastronomía, playas y actividades.
Para cada pregunta se analiza la similitud coseno de los fragmentos recuperados, incluyendo la similitud del primer resultado y la media de los resultados obtenidos.
Estas métricas permiten estudiar el comportamiento de la recuperación semántica, aunque no demuestran por sí solas que la respuesta final sea correcta.

3.2. Evaluación del *function calling*

Se ha definido un conjunto de casos de prueba ***EVAL_CASES*** en el que se especifican las herramientas que deberían utilizarse para cada pregunta. Se contemplan consultas turísticas, meteorológicas, preguntas que requieren ambas herramientas y preguntas generales que no requieren ninguna.
La evaluación compara las herramientas esperadas con las realmente utilizadas También se contemplan casos límite, como fechas meteorológicas fuera del intervalo permitido e información que no aparece en la guía turística.

4. Limitaciones y posibles mejoras

Durante el desarrollo se han identificado las siguientes limitaciones:

- **Dependencia documental:** el asistente únicamente puede fundamentar sus recomendaciones turísticas en la información disponible en la guía.

- **Disponibilidad de servicios externos:** las consultas meteorológicas dependen del funcionamiento de Open-Meteo y de su intervalo de predicción.
- **Limitación geográfica:** las previsiones utilizan las coordenadas de Santa Cruz de Tenerife, por lo que pueden no reflejar las condiciones meteorológicas de otras zonas de la isla.
- **Pérdida de contexto:** el mecanismo de resumen del historial puede descartar información que resulte importante en turnos posteriores.
- **Posibles alucinaciones:** aunque se utiliza RAG y se exige citar fuentes, el modelo todavía puede interpretar incorrectamente los documentos o generar información no suficientemente fundamentada.

Como posibles mejoras futuras se plantea ampliar la base documental, incorporar previsiones para distintas localidades de Tenerife, implementar una interfaz web y realizar una evaluación más extensa de la calidad de las respuestas.

5. Conclusiones

El proyecto integra las principales técnicas estudiadas durante el módulo: recuperación de información mediante RAG, gestión del contexto conversacional y ejecución de herramientas externas mediante *function calling*.

Las decisiones técnicas se han orientado a desarrollar una solución funcional y comprensible, priorizando herramientas conocidas, componentes sencillos y un control explícito sobre el comportamiento del asistente.
La combinación de Gemini, FAISS y Open-Meteo permite construir un asistente que complementa la generación de lenguaje con información documental y meteorológica externa.

El desarrollo también pone de manifiesto la importancia de evaluar no solo las respuestas generadas, sino también la selección de herramientas, el tratamiento de errores y la fiabilidad de las fuentes utilizadas.